

Compute Factorial Approaches

This document summarizes all the approaches we discussed for solving the "Compute Factorial" problem, including optimizations for large inputs and modulo operations.

1. Recursive Approach (Basic)

```
public static long Factorial(long n) {  
    if (n == 0 || n == 1)  
        return 1;  
    return n * Factorial(n - 1);  
}
```

Pros: - Simple and intuitive.

Cons: - Stack overflow for large n. - O(n) time per test case. - Not suitable for $T > 10^4$ or $n > 10^4$.

2. Iterative Approach (Basic)

```
public static long Factorial(long n) {  
    long result = 1;  
    for (long i = 2; i <= n; i++) {  
        result *= i;  
    }  
    return result;  
}
```

Pros: - Avoids recursion. - Easy to understand.

Cons: - Overflow occurs for $n > 20$ when using long. - Slow for multiple test cases with large n.

3. Modular Factorial (Recursive)

```
public static final long MOD = 1_000_000_007;  
  
public static long Factorial(long n) {  
    if (n == 0 || n == 1) return 1;  
    return ((n * Factorial(n - 1)) % MOD) % MOD;  
}
```

Pros: - Correct modulo arithmetic.

Cons: - Still TLE for large n due to recursion.

4. Modular Factorial (Iterative)

```
public static long Factorial(long n) {  
    long result = 1;  
    for (long i = 2; i <= n; i++) {  
        result = (result * i) % MOD;  
    }  
    return result;  
}
```

Pros: - Fast for moderate n. - Avoids recursion.

Cons: - Recomputes factorial for each test case. - TLE for large n and multiple test cases.

5. Precomputation with Array (Optimized for Multiple Queries)

```
int maxN = ...; // max n among all test cases  
long[] fact = new long[maxN + 1];  
fact[0] = 1;  
for (int i = 1; i <= maxN; i++) {  
    fact[i] = (fact[i - 1] * i) % MOD;  
}
```

- Each test case: `fact[n]` $O(1)$ lookup.

Pros: - Fast for multiple test cases. - Total time: $O(\text{maxN} + T)$

Cons: - Requires extra $O(\text{maxN})$ memory.

6. Precomputation without StringBuilder

```
for (int n : queries) {  
    System.out.println(fact[n]);  
}
```

- Direct printing, works fine.

Pros: - Simple.

Cons: - Multiple `println` calls may slow down large output.

7. Precomputation with BufferedWriter (Fast Output)

```
BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(System.out));
for (int n : queries) {
    bw.write(fact[n] + "\n");
}
bw.flush();
```

Pros: - Fastest output method in Java. - Handles huge number of test cases.

8. Optimized Approach Without Arrays (Incremental Computation)

```
long lastComputed = 1;
long lastN = 0;

while (t-- > 0) {
    long n = sc.nextLong();
    for (long i = lastN + 1; i <= n; i++) {
        lastComputed = (lastComputed * i) % MOD;
    }
    lastN = Math.max(lastN, n);
    System.out.println(lastComputed);
}
```

Pros: - No array storage. - No recursion. - $O(\max(n))$ total computation. - Memory-efficient (just a few variables).

Cons: - Works best if test cases are increasing or can be sorted.

Summary of Approaches

Approach	Time Complexity	Space Complexity	Suitable For
Recursive	$O(n)$	$O(n)$ stack	Small n , single test case
Iterative	$O(n)$	$O(1)$	Small/moderate n
Recursive Modulo	$O(n)$	$O(n)$ stack	Small/moderate n , modulo
Iterative Modulo	$O(n)$	$O(1)$	Moderate n , modulo
Precompute Array	$O(\max N + T)$	$O(\max N)$	Multiple queries, fast

Approach	Time Complexity	Space Complexity	Suitable For
BufferedWriter Output	$O(\max N + T)$	$O(\max N)$	Large output, competitive coding
Incremental (No Array)	$O(\max(n))$	$O(1)$	Memory-efficient, multiple queries

These approaches cover all strategies for factorial computation, from naive to highly optimized, both with and without modulo, and with memory considerations.